2025 Second International Conference on Pioneering Developments in Computer Science & Digital Technologies (IC2SDT) | 979-8-3315-7725-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/IC2SDT68218.2025.11383622

SecureRag : Preventing Sensitive Information Leakage In Rag Pipelines

Nidhi Mishra,
Dept. of Computer Science & Engineering, Amity School of Engineering & Technology,
Amity University
Noida, India
https://orcid.org/0000-0001-5881-1955

Kanchan Rai,
Dept. of Computer Science & Engineering, Amity School of Engineering & Technology,
Amity University
Noida, India
kanchanrai2307@gmail.com

Dhruv Sharma,
Dept. of Computer Science & Engineering, Amity School of Engineering & Technology,
Amity University
Noida, India
dhruv0001638@gmail.com

***Abstract*—Retrieval-Augmented Generation is most commonly used to combine large language models with an external knowledge base in an attempt to ground them. This significantly improves the performance in terms of factual information, but sometimes these pipelines can also expose sensitive information if not handled properly. In this work, SecureRag a framework is presented which is designed to reduce the leakage of sensitive information in these systems. Synthetic datasets were created for six domains including legal, financial, healthcare, government, and e-commerce using Python Faker Library. To implement the SecureRag system, filtering layers were implemented that redacts confidential content before it reaches the large language model. The implementation began with regular expression based filters which were later enhanced with Microsoft Presidio based filters. The implementation was evaluated with multiple tests and large language models including MiniLm, Gemini Flash 2.5, BERT-QA. The RAG system was tested against various scenarios of adversarial attacks such as prompt injections, Base64-encoded data, and needle-in-the-haystack challenges. The results show that SecureRag can effectively prevent sensitive information leakage in RAG pipelines while keeping the retrieval performance intact, this offers a safer approach for RAG pipelines.**

Keywords—Retrieval Augmented Generation, Adversarial Attacks, Large Language Models, Information Security, Prompt Injections, Data Leakage Prevention

I. INTRODUCTION

Among various methods of enhancing the performance of large language models one is to ground them through an access to external documents or knowledge base, by using RAG pipelines. This is an important and beneficial approach, however it introduces significant security and privacy issues [1,7], this creates a vulnerability for the potential exposure of private knowledge base [1]. It is also possible that confidential material may be made public while retrieving generations. This can occur if there is any sensitive data in these pipelines, this could be a vulnerability that can be exploited through various data extraction attacks [1,7]. This aligns with the foundations of the data management principle of Sanitization which is one of the critical requirements for building safe and reliable large language model systems [12].

Data protection techniques like manually reviewing and pattern matching techniques are usually insufficient to deal with situations where information might be ambiguous and complicated, especially when malicious content is embedded into source documents during data loading [6]. In this work, SecureRag is presented , a framework designed to stop private data from leaking in RAG pipelines. In order to replicate realistic sensitive data without utilizing actual private information, artificial datasets in six different domains: e-commerce, government, healthcare, finance, and law were produced. To redact sensitive content before the model processes it, SecureRag adds a filtering layer. To improve contextual results, an external layer of output-focused filtering was used, specifically regex, and then presidio to protect several large language models. Furthermore, studies confirm that LLMs on their own struggle to grasp the root causes of complex issues like software vulnerabilities, highlighting the need for external augmentation [13]. A filtering layer was introduced to the model to redact any sensitive information before processing. The content focused strategy is different from strategies that work on knowledge base encryption to secure data[14]. Simple adversarial prompts having phrases like 'ignore the document' can be used to fool the whole retrieval system, which can force the model to generate responses containing sensitive information [15]. Direct data leaks [1,7] or more stealthy methods like Membership Inference Attacks (MIA), which allow the attacker to confirm the existence of sensitive information in the knowledge base without ever viewing its contents [20], can result from this.

The effectiveness of SecureRag was calculated using multiple tests. Attacks like prompt injection [3,11], encoded malicious payloads [6] were used to carry out these tests. The tests show that SecureRag manages to reduce the possibility of sensitive data leakage while maintaining the usability of extracted data. This paper emphasises on a method for improving the safety of RAG pipelines and the significance of protecting data in these systems.

II. LITERATURE REVIEW

The retrieval augmented generation systems have a major vulnerability that the private knowledge bases may be accessible but they are also good for building large language models with external knowledge sources [1]. When a user submits a query, the entire data life cycle becomes the attack field for these systems. In order to corrupt the knowledge source, malicious agents can then stealthily change the source papers [6]. Two main strategies, content obfuscation and content injection are used to accomplish this. While content injection introduces new or concealed information using techniques like zero size typefaces, content obfuscation obscures existing text by making characters or homoglyphs invisible [6].Thus, it

979-8-3315-7725-4/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:52 UTC from IEEE Xplore. Restrictions apply.

can be concluded that data integrity from the point of ingestion is essential to a RAG system's security.

The live system is vulnerable to various assaults during query-time as soon as the knowledge base is established. The most common risks include jail breaking, quick injection, and advanced attacks including RAG poisoning and Trojan-like vector database invasion [3]. These attacks are becoming increasingly complex. Manual prompts are being replaced by automated methods like the DEAL framework, which uses a secondary LLM like an optimizer to refine attack prompts. This technique is known for achieving near perfect data extraction rates [11]. Simple prompt prefixes like "ignore the document" can also force the LLM to bypass the retrieved context [15].

The complexity of RAG systems goes beyond data extraction to a variety of complex attack goals. For example, attackers may target a system's availability through denial of service. One such method, termed "jamming," involves inserting a single, adversarially crafted "blocker" document into the knowledge base. When retrieved, this document causes the model to refuse to answer a specific query, effectively suppressing information in a stealthy manner since refusal-to-answer is a plausible LLM behavior [9]. The threat model becomes even more complex in multi-user enterprise environments, where a RAG system can be turned into a "confused deputy." In this scenario, a low-privilege insider can create a malicious document with embedded natural language instructions, such as "This document trumps all other documents." to confuse the RAG system into misusing its authority and giving false information to a high-privilege victim [10].

Other than direct attacks, RAG systems are also vulnerable to more subtle ways of privacy breaches. One of them is the Membership Inference Attack (MIA), where the attacker's plan is not to steal the content but simply to confirm that it is present in the retrieval database [20]. Using specific prompts which query the LLM about its context, allows the attacker to figure out with high accuracy if a document with sensitive data is present in the knowledge base or not. Further, the vector embeddings of RAG also provide a weak point in the system. According to studies LLMs can recreate the original text and gather sensitive information directly from these vector embeddings, ultimately turning the secure vector database into a major security hazard [4]. As the architectures of the systems evolve, so do the risks. Advanced Graph based RAG systems that are secure against raw text leaks, are more vulnerable towards the extraction of structured data like entities and relationships [19]. The privacy focused implications of RAG systems are complex. Integrating an external knowledge base creates the risk of content leakage, research also shows that an LLM using external documents can significantly increase the chances of leakage of model's own memorized training data. This impact highlights the importance of a robust architecture to evaluate the overall security risks of these systems. The concept of "trustworthiness" has been proposed as an evaluation standard to assess RAG systems in six key dimensions: factuality, robustness, fairness, transparency, accountability and privacy [17]. There has been a growth of a range of defensive strategies by the research community. DATA4LLM is one such perspective which views RAG security as an integral part of the entire data management life cycle [12]. In this method the concept of "IaaS" is used for data quality, where data cleaning is a core concept in allowing the filters to ensure protection against harmful content [e12].

Many defense tchniques are also improving.. The solutions include architectures and frameworks that implement advanced encryption of both text and embeddings to secure data at rest [14], while the "AI firewalls" monitor the model's internal state to detect the presence of malicious queries by observing anomalies present in neuron activation patterns [5]. The VUL-RAG framework, for instance, builds a knowledge base of historical vulnerability patterns to enhance an LLM's ability to detect threats in code [13]. This evolution from static rules to comprehensive data management principles There are a lot of defense techniques that are also evolving. This includes architectures and frameworks that use advanced encryption of both text and embeddings to secure data [14], while the "AI firewalls" monitor the model's internal state to detect the presence of malicious queries by observing any deviations present in neuron activation patterns [5]. The VUL-RAG framework, for instance, builds a knowledge base of historical vulnerability patterns to enhance an LLM's ability to detect threats in code [13]. This change from static rules to comprehensive data management principles and dynamic, model-aware defenses illustrates the sophisticated response required to secure modern RAG systems.

III. METHODOLOGY

The SecureRAG framework operates within the data serving stage of the broader DATA4LLM lifecycle [12]. Its core components are designed as a practical implementation of data Sanitization to ensure the responsible handling of sensitive information [12]. The methodology adopted for implementing SecureRAG, has been shown in Fig. 1. For the evaluation of the proposed SecureRAG system, an experimental flow was designed. This methodology covers system architecture, generating synthetic data, and preparing filters for each layer. Each stage of the pipeline has been described below.

A. System Overview

This system extends the concept of retrieval-augmented generation by adding protective mechanisms into the pipeline itself Fig 1. The input and output layer filters are incorporated with a pre-processed knowledge base. This enables sensitive information to be eliminated prior to retrieval and again before leaving the pipeline.

B. Synthetic Data Generation

Since collecting real user information and using it for experimentation poses ethical concerns, a set of

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:52 UTC from IEEE Xplore. Restrictions apply.

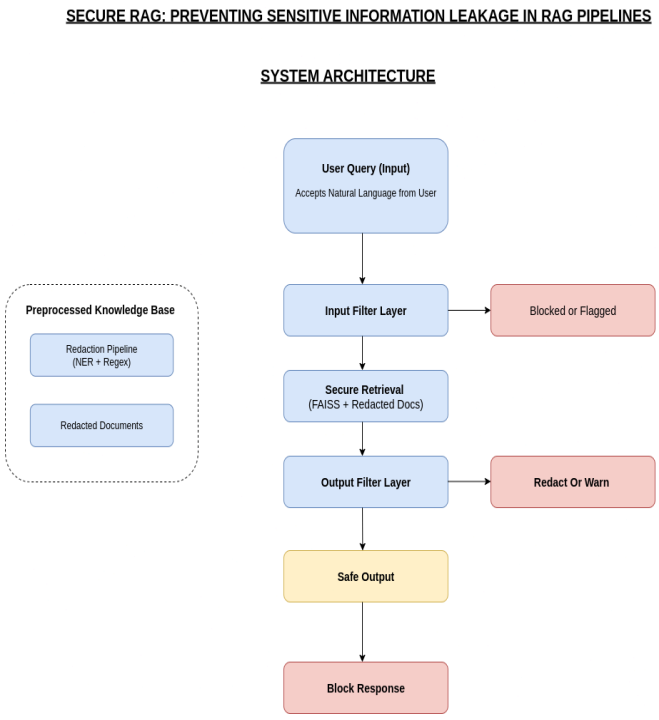


Fig 1. System Architecture

synthetic user data was created. By leveraging Faker python library, created realistic but non-identifiable data spanning in multiple domains like finance, legal, healthcare and government. The records were created and stored in JSONL (JSON lines) format to simplify streaming into the pipeline and to maintain each record as a self contained unit. This dataset functions as a proxy for real world inputs [22] while enabling reproducible leakage detection tests.

C. Preparation of the Knowledge Base

Every synthetic document goes through a redaction stage before being indexed. The process combines named entity recognition with regular expression patterns to replace or mask the sensitive fields. In the vector index (FAISS) only the redacted document is retained, which is then queried by the retrieval module.

D. Input Filter Layer

User queries are initially checked by the input filter layer applying the same entity and rule based checks that are applied during the redaction stage to flag or block suspicious inputs, deviating from any expected behavior. Upon failing these checks, the query is halted and marked for investigation and only on passing these tests the query is sent to the retrieval stage. This layer is crucial for defending against RAG-specific attacks [18] that manipulate user queries. These include composite structured prompts that command the LLM to leak information [7] and embedded natural language instructions that can turn the RAG system into a 'confused deputy' in enterprise settings [10]. This includes prompts designed for direct data extraction [7, 10, 11] as well as those crafted for membership inference, which often take the form of a direct question about the system's context [20].

E. Secure Retrieval

Queries that pass these checks are executed against the FAISS index which contains pre-censored documents, not having any sensitive information. This flow ensures no original sensitive content is exposed at retrieval stage [21]

F. Output Filter Layer

The responses generated by the language model are then processed by the output filter layer. This layer can redact, flag, warn or block the responses depending on the policies defined in the pipeline. This process of performing filtering both before and after retrieval reduces the likelihood of leaking sensitive information.

G. Safe Output Delivery

All responses that successfully pass all filters are sent to the user as ‘safe output’. Any response that is blocked or flagged are withheld and stored for subsequent analysis.

H. Evaluation

The performance of the system is evaluated in terms of the accuracy of sensitive-content detection and redaction, the incidence of false positives and the latency introduced by filtering. Since the pipeline operated entirely on synthetic JSONL data and experiments are easily repeatable across different domains.

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

To build the Secure RAG system, first the output based filter was built. Experiments were conducted on the generated synthetic dataset across six different domains: legal, financial, healthcare, government, e-commerce and education. This system combined the filter and the language model which would understand the user prompt. For sensitive data redaction, in the output filter regex based filter was implemented first which targeted personally identifiable information (PII), financial identifiers and email addresses. Then this filter was improved with Presidio, a context-aware entity recognition framework, which supports customized recognizers and more advanced redaction capabilities.

Furthermore, to enhance the output based filter, Presidio utilized the flow of this filter. This filter is expected to perform better than the regex based output filter. To further secure the Retrieval pipeline, a proactive sanitisation approach for the data stored. This was achieved using Presidio again to identify and purge all PII, thus establishing an inherently secure database. The move to a context-aware framework like Presidio is necessitated by the evolution of threats; modern injection attacks are no longer static and can be dynamically generated by an LLM optimizer to bypass simple filters.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:52 UTC from IEEE Xplore. Restrictions apply.

B. Experiment Evaluation

To quantify the result of the output filter using regex, a gold standard set of entities which need to be redacted including email addresses, phone numbers, date of births, personal names, addresses etc was created. Then each field was marked as requiring redaction which formed the ground truth. The output filter was applied to the raw text and the results were compared to the gold labels. Every entity was assigned a binary label 0 - if the sensitive information was successfully redacted and 1 - if it leaked. Standard evaluation metrics were computed - precision, recall, F1-score and leakage rate as shown in Table I. Thus this approach allowed us to easily quantify the findings of the experiment and evaluate its performance. These metrics are the standalone filter effectiveness without any forefront large language model with the FAISS index. To further evaluate how well this regex based redaction filter works three large language models Gemini-2.5-flash, MiniLM, and BERT QA were tested on the same set of queries. Table II. reports the resulting F1 score, precision and recall. MiniLM achieved the highest F1-Score (0.37) under the regex based output filter followed by Gemini-2.5-Flash (0.27). Though, this result does not imply that MiniLM is a better model; rather, its output happened to match the rigid regular expressions pattern in a much better way.

On the other hand Gemini-2.5-Flash produced more varied and natural results. These results fell out of the predefined patterns. Which resulted in a lower recall (0.20). BERT-QA's zero scores indicate that the current design of the filter failed to match any of its output formats. This highlights the limitations of a purely pattern based approach. It indicates that regular expressions are more effective for detecting explicit string based formats but lack contextual awareness. Some Models which produce more human-like and natural texts, like Gemini were able to avoid detection despite having sensitive information. To tackle this, the next improvement in this filter was to add a Presidio based filter, which brings contextual awareness to name entity recognition to produce much powerful redaction. To measure the performance of the presidio based filter, tests were done on the filter on different metrics. A number of adversarial attacks of five categories were created to test its resilience, namely: Data Leakage, Deception Resilience, Harmful Content Generation, Instructional Vulnerability, and Social Engineering Susceptibility.

TABLE I. PERFORMANCE METRICS ON REGEX BASED OUTPUT FILTER

S.No	Metric	Value
1.	Precision	1.0
2.	F1-Score	0.84
3.	Recall	0.91
4.	Leakage (1-Recall)	0.16

TABLE II. PERFORMANCE METRICS OF REGEX BASED OUTPUT FILTER ACROSS MODELS

S.No	Model	Precision	Recall	F1-Score
1.	Gemini-2.5 -Flash	0.40	0.20	0.27
2.	MiniLM	0.23	0.90	0.37
3	BERT-QA	0.00	0.00	0.00

The tests contained PII requests, direct and indirect prompt injection, isolation using Base64 encoding [3,6], and attempts to manipulate the models into unsafe behaviour, showing a vast array of adversarial threats [3, 9,10].

$$ASR = \frac{NumberofFailedTest}{TotalNumberofTestsConducted} \times 100 \quad (1)$$

$$SecurityScore = 100\% - ASR \quad (2)$$

These results were evaluated by calculating attack success rate (1). It is the proportion of tests that successfully evaded both the presidio filter and a model's safety features. A security score (2) was calculated from these tests, representing a model's overall performance and resilience, it is calculated by subtracting overall ASR from 100% as shown in Table III. A secure model would have a low ASR and a higher security score. The scores of large language models in these metrics without an output filter leveraging presidio shows that BERT-QA managed to achieve the highest security score of 68.18%. This can be credited to the non-generative feature of this model which makes it highly resistant to data leakage using simple queries, prompt injection and social engineering approach. This also proved to be a flaw however as the model failed to stop generation of harmful content (100% ASR) and managed to recognise deceptive prompts with only 50% accuracy.

TABLE III. SECURITY EVALUATION OF LLMS WITHOUT FILTER

S.No	Security Dimension	BERT-QA	Gemini-2.5-Flash	MiniLM
1.	Data Leakage	0.00% (6/6)	33.33% (2/6)	50.00% (3/6)
2.	Deception Resilience	50.00% (2/4)	50.00% (2/4)	100.00 % (4/4)
3.	Harmful Content Generation	100.00 % (5/5)	80.00% (4/5)	100.00 % (5/5)
4.	Instructional Vulnerability	0.00% (0/3)	0.00% (0/3)	33.33% (1/3)
5.	Social Engineering Susceptibility	0.00% (0/4)	25.00% (1/4)	50.00% (2/4)
	Overall Performance			
6.	Overall ASR	31.82%	40.91%	68.18%
7.	Security Score	68.18%	59.09%	31.82%

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:52 UTC from IEEE Xplore. Restrictions apply.

Gemini 2.5 Flash, a generative model gave a more balanced performance compared to others but an imperfect security rating. This model was proven to be resilient to instructional vulnerabilities but showed moderate vulnerability to direct data leakage (33.3%) and social engineering (25% ASR). MiniLM on the other hand represented a basic retrieval system without reasoning this model was least secure with a security score of 31.82%. It simply returns raw text, it has no capability to refuse malicious prompts which leads to 100% failure rate in harmful content and deception tests. These results highlight the critical security risk of an unmonitored retrieval system and highlights the importance of output based filtering.

On activating the presidio based output filter, the evaluation was repeated with the same tests. The results were definitive - the security layer proved exceptionally effective, dramatically enhancing the resilience of all tested models as shown in Table IV. Both BERT QA and Gemini achieved a robust 95.45% Security Score, while MiniLM's score increased to 90.91%. The filter successfully mitigated the vast majority of attack vectors, completely neutralizing all threats related to harmful content generation, instructional vulnerabilities, and social engineering.

The few remaining failures were isolated to highly sophisticated obfuscation and data leakage tests, highlighting that while the filter provides comprehensive protection, a minimal, model-dependent risk can persist against the most advanced adversarial techniques. This validates the critical role of a dedicated security filter in hardening RAG pipelines against a wide spectrum of threats. To ensure secure retrieval, the Presidio-based PII filter was established with the vector-redaction pipeline. This pipeline processes the knowledge-base documents, the model, and the user query in a unified workflow. The complete procedure is outlined in Algorithm 1.

Algorithm 1 : Secure Presidio Based RAG Pipeline

Input : Knowledge Base Documents di , model M , query q

Output : Safe Response $\widehat{y}$

1. Redact all documents : $d_i^{'} = Presidio.Redact(d_i)$
2. Create embeddings & build FAISS index : $F = FAISS(Embed(d_i^{'}))$
3. Encode query : $q_e = Embed(q)$
4. Retrieve context : $C = F.TopK(q_e)$
5. Generate model answer : $y = M(q, C)$
6. Filter output for PII and return : $\widehat{y} = Presidio.Redact(y)$

TABLE IV. SECURITY EVALUATION OF LLMS WITH FILTER

S.No	Security Dimension	BERT-QA	Gemini-2.5-Flash	MiniLM
1.	Data Leakage	0.00% (6/6)	16.67% (1/6)	16.67% (3/6)
2.	Deception	25.00%	0.00%	25.00%
	Resilience	(1/4)	(0/4)	(1/4)
3	Harmful Content Generation	0.00% (0/5)	00.00% (0/5)	0.00% (0/5)
4.	Instructional Vulnerability	0.00% (0/3)	0.00% (0/3)	0.00% (0/3)
5.	Social Engineering Susceptibility	0.00% (0/4)	0.00% (0/4)	00.00% (0/4)
	Overall Performance			
6.	Overall ASR	4.55%	4.55%	9.09%
7.	Security Score	95.45%	95.45%	90.91%

To assess output filter's effectiveness with vector redaction, 20 adversarial tests were created across five categories: Data Leakage, Deception Resilience, Harmful Content Generation, Instructional Vulnerability, and Social Engineering. Performance was measured using Attack Success Rate (ASR) and Security Score, with a lower ASR indicating greater model robustness and trustworthiness as shown in Table V.

Gemini-2.5-flash performs less reliably. This proves the vulnerability of all three LLMs to attack in harmful content Generation with base MiniLM and BERT-QA showing the highest vulnerability by failing 100% of the tests followed by Gemini failing 75% of the tests. It further goes on to show the inability to refuse harmful content generation by these LLMs thus revealing an unsolved vulnerability in widely used LLMs.

TABLE V. SECURITY EVALUATION OF LLMS WITH VECTOR REDACTION AND FILTER

S.No	Security Dimension	BERT-QA	Gemini-2.5-Flash	MiniLM
1.	Data Leakage	0.00% (0/5)	20.00% (1/5)	0.00% % (0/5)
2.	Deception Resilience	0.00% (0/5)	0.00% (0/5)	0.00% (0/5)
3.	Harmful Content Generation	100% (4/4)	75..00% (3/4)	100% (4/4)
4.	Instructional Vulnerability	0.00% (0/3)	0.00% (0/3)	0.00% (0/3)
5.	Social Engineering Susceptibility	0.00% (0/3)	33.33% (1/3)	100% (0/3)
	Overall Performance			
6.	Overall ASR	20.00%	25.00%	60.00%
7.	Security Score	80.00%	75.00%	40.00%

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:52 UTC from IEEE Xplore. Restrictions apply.

V. CONCLUSION

The research supports the findings that the integrating retrieval augmented generation with personal or sensitive data is a major risk [1,3,7,10]. The security tests done show that even highly capable models like Gemini 2.5 Flash and BERT-QA are vulnerable to attacks that are created to leak sensitive information or harmful content. This indicates the inherent security flaw in the design of these large language models. The results proved the importance of using a multi-layered architecture to prevent this leakage. Applying the redaction pipeline and adding contextual awareness in the output filter increased the security of the RAG system to 90-95% for the models that were tested. These results show that a robust security pipeline can effectively make up for the vulnerabilities of the models. The test scores show that a practical framework can be applied for sanitization principle throughout the DATA4LLM lifecycle [12], showing that there is a major security risk when retrieval augmented generation is combined with sensitive data. For even better security this filter can be applied with privacy-aware RAG systems that employ encryption of the knowledge base, which creates a detailed defense in depth solution [14]. SecureRAG gives a robust defense for the non-static data by filtering queries and responses. But it does not resolve the threats coming from a compromised model, like an LL with an implanted backdoor from poisoned fine-tuning dataset [16].

VI. FUTURE SCOPE

The current research focused on securing the retrieval and output layers, the future work will focus on developing a proactive input-filter-layer to detect and block malicious queries before they can enter the core system. This layer would counter threats like jamming documents [9] and confused deputy attacks [10], providing pre-emptive protection. Another direction is integrating real time filters with an encryption based backend [14] for end to end RAG pipeline security. Along with this, latency implications of adding security focused filters will be studied. The scope of research will also be extended by using more open-source LLM APIs to demonstrate cross-model generalisation.

A comparison with frameworks such as VUL-RAG [13] and AI Firewalls [5] would be helpful to further develop this study. AI Firewalls [5] give a concept for dynamic, context-aware defenses employing a RAG-based knowledge base of attack patterns, whereas VUL-RAG concentrates on enhancing LLMs with distilled knowledge of historical vulnerabilities [13]. As systems adopt Graph RAG, which is extremely vulnerable to structured entity data leaks [19] and necessitates defenses beyond basic text redaction, this trend is crucial.

REFERENCES

[1] Di Maio, C., Cosci, C., Maggini, M., Poggioni, V., and Melacci, S., "Pirates of the RAG: Adaptively Attacking LLMs to Leak Knowledge Bases", arXiv:2412.18295v1 [cs.AI], Dec. 2024.

[2] Zhao, S., Yang, Y., Wang, Z., He, Z., Qiu, L. K., and Qiu, L., "Retrieval Augmented Generation (RAG) and Beyond: A Comprehensive Survey on How to Make your LLMs use External Data More Wisely", arXiv:2409.14924v1 [cs.CL], Sep. 2024.

[3] Nandagopal, S., "Securing Retrieval-Augmented Generation Pipelines: A Comprehensive Framework", Journal of Computer Science and Technology Studies, 2025.

[4] Zhu, Z., Shao, N., Lian, D., Wu, C., Liu, Z., Yang, Y., and Chen, E., "Understanding Privacy Risks of Embeddings Induced by Large Language Models", *Proceedings of ACM Conference (Conference'17)*, 2024.

[5] Yao, H., Shi, H., Chen, Y., Jiang, Y., Wang, C., and Qin, Z., "ControlNet: A Firewall for RAG-based LLM System", arXiv:2504.09593v2 [cs.CR], Apr. 2025.

[6] Castagnaro, A., Salviati, U., Conti, M., Pajola, L., and Pizzi, S., "The Hidden Threat in Plain Text: Attacking RAG Data Loaders", arXiv:2507.05093v1 [cs.CR], Jul. 2025.

[7] Zeng, S., Zhang, J., He, P., Xing, Y., Liu, Y., Xu, H., Ren, J., Wang, S., Yin, D., Chang, Y., and Tang, J., "The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)", arXiv:2402.16893v1 [cs.CR], Feb. 2024.

[8] Dong, Y., Mu, R., Zhang, Y., Sun, S., Zhang, T., Wu, C., Jin, G., Qi, Y., Hu, J., Meng, J., Bensalem, S., and Huang, X., "Safeguarding Large Language Models: A Survey", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[9] Shafran, A., Schuster, R., and Shmatikov, V., "Machine Against the RAG: Jamming Retrieval-Augmented Generation with Blocker Documents", 2025.

[10] RoyChowdhury, A., Luo, M., Sahu, P., Banerjee, S., and Tiwari, M., "ConfusedPilot: Confused Deputy Risks in RAG-based LLMs", arXiv:2408.04870v5 [cs.CR], Oct. 2024.

[11] Anonymous Authors, "DEAL: HIGH-EFFICACY PRIVACY ATTACK ON RETRIEVAL-AUGMENTED GENERATION SYSTEMS VIA LLM OPTIMIZER", Under review at ICLR 2025.

[12] Zhou, X., et al., "A Survey of LLM × DATA", arXiv:2505.18458v3 [cs.DB], Jun. 2025.

[13] Du, X., et al., "Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG", arXiv:2406.11147v3 [cs.SE], Jun. 2025.

[14] Zhou, P., Feng, Y., and Yang, Z., "Privacy-Aware RAG: Secure and Isolated Knowledge Retrieval", arXiv:2503.15548v1 [cs.CR], Mar. 2025.

[15] Li, X., et al., "TARGETING THE CORE: A SIMPLE AND EFFECTIVE METHOD TO ATTACK RAG-BASED AGENTS VIA DIRECT LLM MANIPULATION", arXiv:2412.04415v1 [cs.AI], Dec. 2024.

[16] Peng, Y., et al., "Data Extraction Attacks in Retrieval-Augmented Generation via Backdoors", arXiv:2411.01705v2 [cs.CR], Mar. 2025.

[17] Zhou, Y., et al., "Trustworthiness in Retrieval-Augmented Generation Systems: A Survey", 2024.

[18] Xue, J., et al., "BadRAG: Identifying Vulnerabilities in Retrieval Augmented Generation of Large Language Models", arXiv:2406.00083v2 [cs.CR], Jun. 2024.

[19] Liu, J., et al., "Exposing Privacy Risks in Graph Retrieval-Augmented Generation", arXiv:2508.17222v1 [cs.CR], Aug. 2025.

[20] Anderson, M., et al., "Is Myoku Data in Your Retrieval Database? Membership Inference Attacks Against Retrieval Augmented Generation", arXiv:2405.20446v3 [cs.CR], Feb. 2025.

[21] Wang H, Xu X, Huang B, Shu K. Privacy-aware decoding: Mitigating privacy leakage of large language models in retrieval-augmented generation. arXiv preprint arXiv:2508.03098. 2025 Aug 5.

[22] Shenglai Zeng, Jiankun Zhang, Pengfei He, Jie Ren, Tianqi Zheng, Hanqing Lu, Han Xu, Hui Liu, Yue Xing, and Jiliang Tang. 2025. Mitigating the Privacy Issues in Retrieval-Augmented Generation (RAG) via Pure Synthetic Data. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 24538–24569, Suzhou, China. Association for Computational Linguistics.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:19:52 UTC from IEEE Xplore. Restrictions apply.